

GitLab

Scanner



ABOUT



GIT-CLI-SCANNER

A Git-CLI-Scanner is a command-line tool designed to scan Git repositories for potentially sensitive, private, or accidentally exposed information. It can help developers identify security issues before code is pushed or shared publicly.



Indicator	Level	Color	Meaning
●	HIGH	Red	Hardcoded secrets that must be removed
●	MEDIUM	Yellow	Risky files not in .gitignore
○	IGNORED	Dim	Test/example values (auto-detected)

PURPOSE

The purpose of the Git-CLI-Scanner is to perform automatic scans of Git repositories in order to detect potentially sensitive information that might have been committed by mistake. It is used to help find credentials, API keys, tokens, passwords, configuration files, and other data that can pose a potential security risk if they get into the wrong hands. The scanner allows scanning of Git repositories for secrets using predefined rules and detecting potentially sensitive data that may be present in the history of changes even if it is no longer present in the current version of the code. It aims to prevent human error, ensure repository security, facilitate secure coding practices, and raise awareness of data exposure risks.

CREATOR

Git-CLI-Scanner is designed with a modular architecture, where separate parts handle specific tasks. The first step is executed by the Command-Line Interface that takes the repository path and scanning options from the user. Then, the Repository Analyzer checks the target and collects Git information about it. The File Scanner finds all the files that need to be processed and those that can be excluded from the scan, such as binary files. Next, the Detection Engine is responsible for matching patterns and rules to find potential results. Moreover, the Context and Entropy Analysis module cross-checks results to improve the results and reduce false-positive entries. The Git History Scanner helps to detect previous exposure of the sensitive information that was detected in a repository. After that, the Filtering and Deduplication module eliminates duplicates and irrelevant results. Then, the Severity Analyzer scores the results to help understand how critical the issue is. Finally, the Redaction and Report Generation module prepares and displays the scan result as a terminal output. Overall, these modules form a complete command-line tool that allows for analyzing Git repositories for secrets efficiently.

```
• HIGH · generic-api-key
  File:  t.ts:72
  Match: const issues3 = await scanContent('A

• HIGH · generic-api-key
  File:  t.ts:73
  Match: const issues4 = await scanContent('p

• HIGH · generic-api-key
  File:  t.ts:74
  Match: const issues5 = await scanContent('p

• HIGH · aws-access-key
  File:  t.ts:20
  Match: const issues = await scanContent('A

• HIGH · aws-secret-key
  File:  t.ts:25
  Match: const issues = await scanContent('aw

• HIGH · gcp-api-key
  File:  t.ts:30
  Match: const issues = await scanContent('A

• HIGH · rsa-private-key
  File:  t.ts:65
  Match: const issues = await scanContent('--
```


WORKING PRINCIPLE

The Git-CLI-Scanner is designed as an automated utility that scans a repository for possible exposed secrets. First, it needs to be initialized by a user who inputs a particular command in the command line in which the tool should be launched. The repository path is specified in the command line by the user. The tool checks the repository and lists all files and folders to be scanned. The files that are irrelevant, ignored, or contain binary data are excluded from the scan. Then, the contents of the relevant files are read, and matching rules, patterns, and keywords are applied. The tool helps detect hardcoded credentials, API keys, tokens, passwords, configuration data, or other values that should not be stored in the repository. For improved accuracy, the scanning tool analyzes the context around the matched patterns and uses entropy to spot secrets-like values with high probability.

```
git cli test>git-cli-scanner
i info Setting up pre-commit hook...
✓ success Pre-commit hook installed!
i info Every 'git commit' will now automatically scan your repository

git cli test>git commit -m "test"
i info Git CLI Scanner running...
✗ Found 9 potential vulnerabilities! (9 blockers)

Scan Results:

• HIGH · generic-api-key
  File: t.ts:70
  Match: const issues1 = await scanContent('api_key: "1234567890"')

• HIGH · generic-api-key
  File: t.ts:71
  Match: const issues2 = await scanContent('api-key = "1234567890"')

• HIGH · generic-api-key
  File: t.ts:72
  Match: const issues3 = await scanContent('API_KEY: "1234567890"')

• HIGH · generic-api-key
  File: t.ts:73
  Match: const issues4 = await scanContent('password = "1234567890"')

• HIGH · generic-api-key
  File: t.ts:74
  Match: const issues5 = await scanContent('password = "1234567890"')
```

Apart from scanning the files in the current repository, the Git CLI Scanner can search the Git commits history for possibly sensitive information that was committed before but isn't included in the latest version of the draft. It processes the relevant commits and changes in the files in the same way, following the detection rules and keeping track of the results. Then, the scanner keeps only positive results by using ignore rules, allow listing, placeholder detection, and contextual analysis. Then, it groups repeated results and assigns a severity level for each issue detected depending on factors like information type, confidence of the match, and the location of the data. To guarantee safety, information found will be hidden or made unrecognizable visually. Finally, the scanner asks the user whether they want to continue or update the code, and also provides the fix.

PROCESS FLOW

The Trigger (git commit): When a developer runs `git commit -m "update"`, Git pauses the commit process to check for any configured hooks.

The Interception (pre-commit hook): Git finds our pre-commit hook (installed via the `git-cli-scanner init` command). Instead of completing the commit, Git executes the hook script which runs: `npx git-cli-scanner scan --hook`

The Analysis (Scanning Staged Files): The CLI tool wakes up and analyzes only the files that are currently staged for the commit. It processes these files through a multi-layered security engine:

- Regex Pattern Matching
- Contextual Scanning
- File Path Checking

The Verdict & Action: Based on the scan results, the tool takes one of two paths:

- Clean: If no vulnerabilities are found, the tool silently exits and Git completes the commit instantly.
- Vulnerabilities Found: The tool halts the process, prints a detailed report (showing the exact file, line number, severity, and suggested fix), and prompts the developer: `? Vulnerabilities were found! Do you still want to continue with the commit? (y/N)`

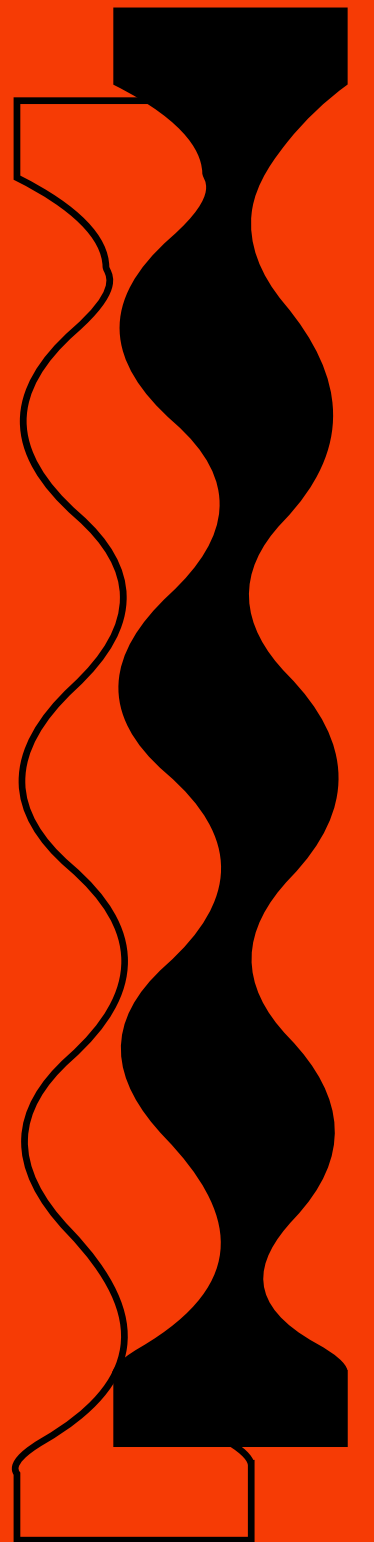
The Final Outcome:

- If the developer answers Yes, the tool allows the commit to proceed despite the warnings.
- If the developer answers No (default), the tool throws an exit code 1. Git sees this failure code and completely aborts the commit. The sensitive data never touches the git history.

<code>scan-all [dir]</code>	Scan an entire directory recursively
<code>scan-all [dir] --show-sol</code>	Scan a directory and show suggested fixes
<code>explore</code>	Launch interactive file explorer TUI

FEATURES

The Git-CLI-Scanner has a lot of valuable capabilities for analyzing the security of Git repositories. One of its key capabilities is its ability to automate the scanning of the source code and configuration files for the purpose of identifying sensitive information according to certain pre-defined rules and pattern matching. The scanner is able to detect any suspicious credentials, passwords, access tokens, etc. It can scan the history of Git commits in order to find out if there are any potentially leaked data. The scanner is highly capable of filtering files, as well as excluding certain directories, analyzing the context of the found strings, analyzing entropy of random strings, filtering out some false positives, eliminating duplicates, and classifying the severity of detection. The scanner masks or redacts sensitive information in its output and generates a structured report, showing all necessary information.



EXISTING SOLUTIONS

1) GitGuardian

- Detects exposed credentials, API keys, tokens, and other sensitive information.
- Provides monitoring and security workflows for exposed secrets.

2) Gitleaks

- Open-source secret scanner designed specifically for Git repositories.
- Uses pattern-based detection to identify potential secrets in source code and Git history.

3) GitHub Secret Scanning & Push Protection

- Automatically identifies supported secret patterns in repositories.
- Push Protection can prevent detected secrets from being pushed to GitHub.

PROBLEMS WE AIM TO ADDRESS

1) False Positives vs. False Negatives

- Pattern-based scanners may flag harmless test/example values or miss unusual real secrets.
- Our approach uses context-aware, multi-layer analysis to improve detection confidence.

2) Detection ≠ Remediation

- Existing scanners can identify a secret, but developers still need to determine why it is risky and how to fix it.
- Our scanner provides a risk explanation and recommended solution.

3) Secrets in Git History

- Removing a secret from the latest version does not remove it from previous commits.
- Our scanner can perform Git-history exposure analysis after remediation.

4) Alert Fatigue

- Multiple findings can overwhelm developers and make critical issues harder to identify.
- Our scanner uses severity and risk prioritization rather than treating every alert equally.

1. Context-Aware Detection
 - Goes beyond simple regex/pattern matching.
 - Considers surrounding code and multiple indicators to determine whether a finding is likely to be a genuine secret.
2. Risk Prioritization
 - Classifies findings by severity and confidence.
 - Critical and high-risk findings are presented first so developers know what requires immediate attention.
3. Fix Recommendation Engine
 - Doesn't stop at “Secret detected.”
 - Explains what was detected, why it is dangerous, and how to remediate it, including secure alternatives.
4. Before & After Verification
 - Re-scans after the developer fixes an issue.
 - Confirms whether the sensitive information has actually been removed.
5. Git History Exposure Analysis
 - Checks whether a removed secret still exists in previous commits.
 - Alerts the developer when revocation, rotation, or history cleanup may be necessary.
6. Custom Detection Rules
 - Allows organizations to define rules for their own keys, identifiers, credentials, and sensitive data.
 - Makes the scanner adaptable to different projects and security requirements.
7. Smart Interactive CLI
 - Shows secret type, file, line number, severity, confidence, masked value, risk, and recommended action.
 - Organizes findings so developers can review them efficiently.

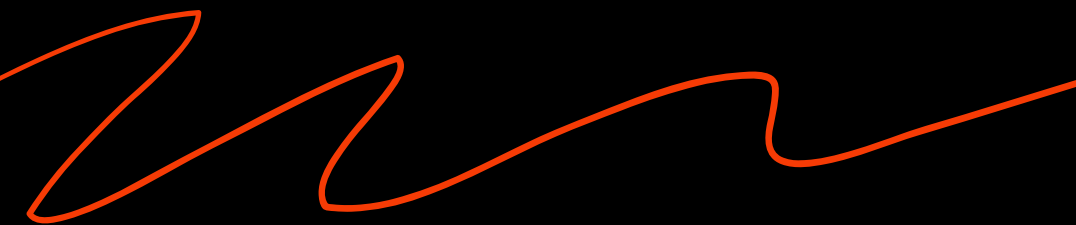
HOW OUR SCANNER IS DIFFERENT



The Git-CLI-Scanner tool has various applications in software development, cybersecurity, and repository management areas. The tool is primarily used to detect and identify any sensitive information like passwords, API keys, access tokens, secret credentials, database connectivity strings, and other confidential or secret data that might have accidentally been stored or committed to Git repositories. As an individual developer, one would use the tool to perform self-code reviews to scan and expose secret information that might have been inadvertently uploaded to a public repository. Developers could also deploy the tool as part of the code-review process and repository-audit workflows to automatically scan multiple files as opposed to manually reviewing each file's contents.

The tool can also be used to conduct Git history scans. This feature allows the user to detect sensitive information that might have been removed from the repository but still exists in the history of the commits. This feature is useful in ensuring that no confidential information accidentally exists in the repository history. This feature would be useful while performing security audits and investigations. Finally, the tool can be used to deploy automated scanning procedures within CI/CD and DevSecOps tooling, where it can help automatically analyze repositories, including performing regular expression analysis, scanning files, filtering false positives, assigning severity levels, and reporting production-grade secrets. Individuals could use the Git CLI Scanner tool as a learning tool to demonstrate how secrets are identified and reported in practice. Organizational uses include migration and audit procedures of repositories, reviewing open-source projects, and ensuring compliance and security assessments. The Git CLI Scanner tool's primary end use is to offer an automated and repeatable scanning procedure that can help improve developer practices and expose some of the areas where poor development practices could lead to inadvertent exposure of confidential data.

APPLICATIONS



THANK
For your attention
YOU

